

GOVERNMENT SCHOOL MANAGEMENT SYSTEM

Development Methodology

Project Execution, Process & Quality Assurance Plan

Document ID: GSMS-MET-001

Version: 1.0

Date: 25 July 2026

Status: Draft for Review

1. Introduction

This document defines how the Government School Management System (GSMS) will be planned, built, tested and delivered. It complements the Software Requirements Specification (GSMS-SRS-001) and Software Design Specification (GSMS-SDS-001) by describing the process, roles, tools, timeline, risk management and quality assurance approach used to turn those specifications into a working system.

2. Development Methodology

2.1 Chosen Approach: Agile Scrum

GSMS will be developed using an Agile Scrum methodology with two-week sprints. Given the number of distinct functional modules and multiple stakeholder groups (school administration, teachers, education authorities), Scrum allows incremental delivery, early stakeholder feedback on real workflows (e.g. attendance, exam entry), and the flexibility to re-prioritize modules such as Welfare or Communication as school-side requirements are clarified during implementation.

2.2 Why Agile Over Waterfall

Consideration	Waterfall	Agile Scrum (Selected)
Requirement stability	Assumes requirements fixed upfront	Accommodates evolving school/Ministry input
Stakeholder feedback	Only at end of each phase	Every 2 weeks via sprint review/demo
Risk exposure	Integration/design risk surfaces late	Surfaces early through incremental builds
Delivery of value	Single release at project end	Usable increments each sprint (e.g. Student & Attendance modules early)

2.3 Roles and Responsibilities

Role	Responsibility
Product Owner	Owns and prioritizes the backlog against school/Ministry needs; accepts completed work.
Scrum Master	Facilitates ceremonies, removes blockers, protects the team's sprint focus.
Development Team	Full-stack engineers implementing frontend, backend and database work per module.
QA Engineer	Designs and executes test cases, manages the defect log, coordinates UAT.
UI/UX Designer	Produces wireframes/mockups per module ahead of sprint implementation.
School Domain Advisor(s)	Principal/Administrator representatives validating workflow accuracy each sprint review.

2.4 Scrum Ceremonies

Ceremony	Frequency	Purpose
Sprint Planning	Start of each sprint (2 weeks)	Select and estimate backlog items for the sprint.
Daily Stand-up	Daily, 15 min	Progress, plans, blockers.
Sprint Review / Demo	End of each sprint	Demonstrate working increment to Product Owner and Domain Advisors.
Sprint Retrospective	End of each sprint	Reflect on process and identify improvements.
Backlog Refinement	Mid-sprint	Clarify and estimate upcoming backlog items.

3. Project Phases and Timeline

The project is organized into five phases across an estimated 20-week delivery timeline for the initial single-school release. Durations assume a core team of 4–6 engineers plus one QA and one designer, and may be adjusted after Sprint 0 estimation.

Phase	Duration	Key Activities	Output
0. Inception	Week 1–2	Requirements sign-off (SRS), architecture sign-off (SDS), environment setup, backlog creation.	Approved SRS/SDS, project backlog, dev/staging environments
1. Foundation	Week 3–5	Auth & RBAC, User Management, base schema, CI/CD pipeline, Student & Teacher Management.	Login, user administration, core student/teacher records
2. Core Academic Modules	Week 6–11	Academic Management, Attendance, Timetable, Examination modules.	Attendance capture, timetable publishing, marks entry & report cards
3. Operational Modules	Week 12–16	Library, Inventory, Welfare, Communication, Reports.	Full module set feature-complete
4. Hardening & UAT	Week 17–19	System/regression testing, performance testing, security review, User Acceptance Testing with school staff.	UAT sign-off, defect closure
5. Deployment & Handover	Week 20	Production deployment, data migration/import, training, documentation handover.	Live system, trained users, support handover

3.1 Sprint Breakdown (Indicative)

Sprint	Weeks	Focus Modules
Sprint 0	1–2	Inception, environment, architecture sign-off
Sprint 1	3–4	User Management, Auth/RBAC, Student Management (core)
Sprint 2	5	Teacher & Staff Management
Sprint 3–4	6–9	Academic Management, Attendance, Timetable
Sprint 5	10–11	Examination & Report Cards
Sprint 6	12–13	Library & Inventory
Sprint 7	14–15	Welfare & Communication
Sprint 8	16	Reports & cross-module dashboards
Sprint 9	17–19	Hardening, security review, UAT
Sprint 10	20	Deployment, migration, training

4. Tools and Technologies

Category	Tool / Technology
Frontend	React.js, Tailwind CSS
Backend	Node.js, Express.js
Database	PostgreSQL
Auth	JWT, RBAC middleware
Version Control	Git, with a trunk-based feature-branch workflow and pull-request review
CI/CD	Automated pipeline running lint, unit tests and build on every pull request; automated deploy to Staging on merge to main
Project Management	Sprint backlog and board (e.g. Jira/Azure DevOps/Trello-class tool)
API Documentation	OpenAPI/Swagger specification generated and versioned alongside the codebase
Testing	Jest/Mocha for unit tests, Supertest for API integration tests, Cypress/Playwright for end-to-end tests
Monitoring	Centralized application logging and uptime/error monitoring in Staging and Production

5. Testing Strategy

Quality is validated through a layered testing strategy aligned to the requirement IDs defined in the SRS, ensuring every functional requirement (FR-<Module>-<n>) maps to at least one automated or manual test case.

Level	Scope	Owner	Timing
Unit Testing	Service and utility functions in isolation	Developers	Continuous, per pull request
Integration Testing	API endpoints against a test database	Developers / QA	Continuous, per pull request
System Testing	End-to-end workflows across modules (e.g. mark attendance → trigger notification)	QA	Per sprint / pre-release
Performance Testing	Load testing key endpoints against the NFR performance targets	QA	Phase 4 (Hardening)
Security Testing	Auth/RBAC boundary testing, input validation, dependency vulnerability scan	QA / Security reviewer	Phase 4 (Hardening)
User Acceptance Testing (UAT)	Real workflows executed by Principal/Admin/Teacher representatives on Staging	School Domain Advisors + QA	Phase 4, prior to go-live

5.1 Defect Management

Defects are logged with severity (Critical, High, Medium, Low) and linked to the originating requirement ID. Critical and High defects block release sign-off; Medium/Low defects may be scheduled into a subsequent sprint by agreement with the Product Owner.

5.2 Definition of Done

- Code merged to main via a reviewed pull request with passing CI checks.
- Unit and integration tests written and passing for the feature.
- Corresponding SRS requirement ID(s) satisfied and traceable in the test plan.
- No open Critical/High defects against the feature.
- Feature demonstrated and accepted at Sprint Review.

6. Risk Management

Risk	Likelihood	Impact	Mitigation
Incomplete/inaccurate baseline student-staff data for import	Medium	High	Early data-quality review during Inception; validation tooling for bulk import.
Limited connectivity at some school sites	Medium	Medium	Pagination, lightweight payloads, and future offline-tolerant attendance capture.
Scope creep across 13 modules	Medium	High	Locked MVP backlog per SRS priority key; changes routed through Product Owner and backlog refinement.
Delayed stakeholder availability for UAT	Medium	Medium	UAT window scheduled and confirmed with school leadership at project kickoff.
Security/data-privacy gaps for minors' data	Low	High	Security design baked into SDS; dedicated security testing phase before go-live.
Key personnel turnover	Low	Medium	Documentation-first culture, pair programming on critical modules, knowledge-sharing in retrospectives.

7. Quality Assurance and Governance

- Weekly project status review against the sprint/phase plan, with burndown tracked at the Scrum Master level.
- Mandatory code review (minimum one approving reviewer) before merge.
- Traceability matrix maintained linking SRS requirement IDs → SDS design elements → test cases, updated each sprint.
- Release sign-off requires: passing CI, zero open Critical/High defects, and Product Owner acceptance at Sprint Review.
- Post-deployment: a 2-week hypercare period with priority defect response before transitioning to standard support.

8. Training and Handover

Ahead of go-live, role-specific training sessions will be conducted for Administrators, Teachers and Librarians, supported by short how-to guides per module. A System Administrator handbook will cover user provisioning, data import, backup verification and first-line troubleshooting. Source code, API documentation (OpenAPI spec), the database schema (per GSMS-SDS-001) and this methodology document will be handed over as the permanent technical record of the project.